



UNIVERSITÀ DEGLI STUDI DI FIRENZE
INGEGNERIA INFORMATICA

Implementazione di Statistiche d'Ordine Dinamiche

Autore:
Neri Saldutti

Corso:
Algoritmi e Strutture Dati

N° Matricola:
7171388

Docente:
Simone Marinai

a.a. 2025/2026

Indice

1	Introduzione	1
1.1	Obiettivo	1
1.2	Specifiche delle condizioni sperimentali	1
2	Fondamenti teorici	2
2.1	Statistiche d'ordine dinamiche	2
2.2	Prestazioni attese	2
3	Documentazione	4
3.1	Architettura del codice	4
3.2	Implementazione delle strutture dati	4
3.3	Implementazione degli esperimenti	7
4	Esperimenti e risultati	9
4.1	Scelta delle quantità	9
4.2	Presentazione dei risultati	9
5	Conclusioni	15

Elenco delle figure

1	Diagramma UML delle tre implementazioni	7
2	Andamento con ingresso lineare	10
3	Andamento con ingresso casuale	11
4	Andamento eliminazione casuale	12
5	Andamento ranking casuale	13
6	Andamento selezione casuale	14

Elenco delle tabelle

1	Complessità teorica di una lista ordinata	2
2	Complessità teorica di un ABR con statistiche d'ordine	3
3	Complessità teorica di un AVL con statistiche d'ordine	3

1 Introduzione

1.1 Obiettivo

La presente relazione ha l'obiettivo di descrivere l'implementazione e l'analisi sperimentale delle *statistiche d'ordine dinamiche*, sviluppate in linguaggio *Python* mediante l'estensione di tre strutture dati distinte: una *lista ordinata*, un *albero binario di ricerca* (ABR) e un *albero AVL*.

Le tre implementazioni sono state sottoposte a diversi tipi di test, al fine di analizzare l'andamento dei tempi di esecuzione ottenuti e confrontarli con le complessità asintotiche attese dall'analisi teorica.

1.2 Specifiche delle condizioni sperimentali

Si riportano le specifiche hardware del calcolatore su cui sono stati svolti i test.

- **CPU:** AMD Ryzen 5 5600H 6 core 12 threads
- **GPU:** AMD Radeon Graphics Vega 7 e NVIDIA GeForce RTX 3050
- **SSD:** SSD M.2 PCIe NVMe 512 GB
- **RAM:** 16 GB DDR4-3200

Il codice è stato eseguito utilizzando **Python 3.14.5** e l'IDE **PyCharm 2026.1.2** su sistema operativo **Arch Linux**. La relazione è stata redatta utilizzando $\text{\TeX}$ e $\text{\LaTeX}$ l'editor offline **TeXStudio v4.9.3**. Gli schemi UML sono stati realizzati utilizzando **Mermaid**. I file Python sono stati realizzati utilizzando esclusivamente la lingua **inglese**.

2 Fondamenti teorici

2.1 Statistiche d'ordine dinamiche

Le strutture dati aumentate, estendono una struttura dati esistente, ad esempio aggiungendo nuovi attributi ai suoi nodi, in modo da poter implementare in maniera efficiente nuovi metodi.

Le statistiche d'ordine dinamiche sono delle particolari strutture dati aumentate che consentono di introdurre due funzionalità principali:

- `OS-SELECT(i)` che ritorna il nodo con l'*i-esima* chiave più piccola,
- `OS-RANK(x)` che ritorna il rango di un determinato nodo rispetto ad un attraversamento ordinato della struttura.

Questa relazione si concentra in particolare sull'analisi delle operazioni di *inserimento*, *eliminazione*, *ranking* e *selezione*.

2.2 Prestazioni attese

Le prestazioni delle operazioni considerate dipendono dalla struttura dati sottostante e delle scelte implementative adottate. Di seguito vengono analizzate le diverse implementazioni, concentrandosi in particolare sul caso peggiore, il caso medio e il caso migliore.

Lista Ordinata

Una lista ordinata è una struttura dati che mantiene i suoi elementi in ordine, in questo caso crescente. Il costo delle singole operazioni dipende dalla posizione dell'elemento su cui si vuole operare e, di conseguenza, dalla modalità con cui si implementa la ricerca di un elemento. Assumiamo una ricerca lineare.

Considerata una lista di n elementi, nel caso migliore l'operazione coinvolge solamente il primo elemento della struttura e si avrà quindi una complessità di $\Theta(1)$. Nel caso peggiore, invece, è necessario scorrere fino all'ultimo elemento e la complessità sarà dunque $\Theta(n)$.

L'analisi del caso medio richiede invece un'analisi probabilistica. Assumendo che i dati in input siano distribuiti in maniera uniforme, in media sarà necessario scorrere circa $\frac{n}{2}$ elementi, risultando in una complessità di $\Theta(n)$.

Per le specifiche operazioni si rimanda alla Tabella 1.

	Caso peggiore	Caso medio	Caso migliore
Inserimento	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$
Eliminazione	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$
Ricerca	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$
Ranking	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$
Selezione	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$

Tabella 1: Complessità teorica di una lista ordinata

Albero Binario di Ricerca

Un albero binario di ricerca, comunemente abbreviato con ABR o, dall'inglese, BST (*Binary Search Tree*), è una struttura dati costituita da nodi organizzati gerarchicamente. Ogni albero ha un nodo detto *radice* (*root*) che rappresenta l'inizio della struttura.

Ogni nodo presenta tre puntatori: uno al figlio destro, uno al figlio sinistro e uno al nodo genitore, organizzati in modo tale che tutte le chiavi appartenenti al sottoalbero sinistro di un nodo siano minori della chiave del nodo stesso, mentre tutte le chiavi appartenenti al sottoalbero destro siano maggiori.

La complessità di un ABR dipende dunque dalla sua struttura interna e in particolare dalla sua altezza h , definita come il numero di archi presenti nel cammino più lungo dalla radice a una foglia (ossia un figlio nullo). L'altezza è a sua volta influenzata dall'ordine di inserimento degli elementi. Nel caso peggiore ogni nodo avrà solamente un figlio e si avrà dunque $h = n - 1$. Nel caso migliore invece si avrà un albero perfettamente bilanciato e l'altezza sarà $h = \lg(n)$.

Per le specifiche operazioni si rimanda alla Tabella 2.

	Caso peggiore	Caso medio	Caso migliore
Inserimento	$\Theta(n)$	$\Theta(\lg n)$	$\Theta(1)$
Eliminazione	$\Theta(n)$	$\Theta(\lg n)$	$\Theta(1)$
Ricerca	$\Theta(n)$	$\Theta(\lg n)$	$\Theta(1)$
Ranking	$\Theta(n)$	$\Theta(\lg n)$	$\Theta(1)$
Selezione	$\Theta(n)$	$\Theta(\lg n)$	$\Theta(1)$

Tabella 2: Complessità teorica di un ABR con statistiche d'ordine

Albero AVL

Un albero AVL è un albero binario di ricerca bilanciato in cui, per ogni nodo, le altezze dei sottoalberi sinistro e destro differiscono al più di un'unità.

In un AVL, l'altezza di un nodo è definita come il numero di archi presenti nel cammino più lungo che collega il nodo a una foglia. Grazie al vincolo di bilanciamento, si può dimostrare che un AVL contenente n nodi ha altezza $h \leq 2 \lg(n + 1)$. Di conseguenza, tutte le operazioni che non modificano la struttura dell'albero presentano complessità logaritmica.

Per mantenere tale proprietà di bilanciamento, gli alberi AVL introducono particolari operazioni di ristrutturazione, dette *rotazioni*. In particolare, si distinguono le operazioni *LEFT-ROTATE*, che effettua una rotazione verso sinistra, e *RIGHT-ROTATE*, che effettua una rotazione verso destra.

Lo scopo di queste operazioni è mantenere il *Fattore di Bilanciamento* (*Balance Factor*) di ogni nodo, calcolato come differenza delle altezze tra il sottoalbero destro e sinistro, compreso tra -1 e 1 .

Per le complessità delle specifiche operazioni si rimanda alla Tabella 3.

	Caso peggiore	Caso medio	Caso migliore
Inserimento	$\Theta(\lg n)$	$\Theta(\lg n)$	$\Theta(1)$
Eliminazione	$\Theta(\lg n)$	$\Theta(\lg n)$	$\Theta(1)$
Ricerca	$\Theta(\lg n)$	$\Theta(\lg n)$	$\Theta(1)$
Ranking	$\Theta(\lg n)$	$\Theta(\lg n)$	$\Theta(1)$
Selezione	$\Theta(\lg n)$	$\Theta(\lg n)$	$\Theta(1)$

Tabella 3: Complessità teorica di un AVL con statistiche d'ordine

3 Documentazione

3.1 Architettura del codice

Il codice di laboratorio è diviso in tre sezioni principali:

- Implementazione delle strutture
- Implementazione dei test da effettuare
- Implementazione del *plotter* dei grafici

La sezione relativa all'implementazione delle strutture dati, include tre file principali, ognuno dei quali contiene due classi: una per il nodo costituente e una per la struttura vera e propria. Il file `os_ordered_list.py` contiene l'implementazione della statistica d'ordine dinamica mediante lista ordinata, `os_bst.py` quella basata sull'albero binario di ricerca e `os_avl.py` l'implementazione tramite albero AVL.

Il file `test_runner.py` contiene l'implementazione di tutti i test che devono essere effettuati e contiene inoltre, la funzione `main()`. Infine, il file `plot_results.py` si occupa del salvataggio e lettura dei dati grezzi in formato `.csv`, oltre che alla generazione di ogni singolo grafico in formato `.pdf`.

3.2 Implementazione delle strutture dati

Per il diagramma UML completo delle classi fare riferimento alla Figura 1.

Lista Ordinata

Per l'implementazione tramite lista ordinata si crea una lista doppiamente collegata. Di fatto ogni `OrderedListNode` avrà tre attributi fondamentali: la chiave `key`, il puntatore al nodo successivo `next` e il puntatore al nodo precedente `prev`. Il nodo in testa alla lista avrà `prev = null`, mentre l'ultimo nodo della coda avrà `next = null`.

Di sotto si riporta la spiegazione dei metodi della lista ordinata.

- `insert(key)`: inserisce in maniera ordinata un nuovo elemento di chiave `key` nella lista.
- `delete(key)`: rimuove il primo elemento di chiave `key` dalla lista. Se l'elemento non è presente scorre tutta la lista. In seguito all'eliminazione vengono aggiustati i puntatori dei nodi adiacenti a quello eliminato in maniera corretta.
- `print(key)`: scorre la lista e stampa ogni elemento in ordine.
- `select(i)`: restituisce la chiave dell'*i*-esimo nodo all'interno della lista
- `rank(key)`: restituisce la posizione a partire dalla testa della lista (rango) del nodo di chiave `key`.

Albero Binario di Ricerca

Il singolo nodo dell'ABR, `BSTNode`, è stato implementato mediante i seguenti attributi: la chiave `key`, il puntatore al figlio destro `right`, il puntatore al figlio sinistro `left` e il puntatore al nodo genitore `parent`. Il nodo radice, `root`, è l'unico nodo per cui il puntatore `parent` risulta nullo.

Per l'ABR sono stati implementati i seguenti metodi:

- `insert(key)`: inserisce l'elemento di chiave `key` all'interno dell'albero nella posizione corretta.
- `delete(key)`: rimuove l'elemento di chiave `key` dall'albero effettuando in seguito una ristrutturazione per ricollegare i nodi in maniera corretta. Fa uso del metodo `successor()`.
- `print(key)`: metodo implementato per avere la stessa interfaccia fra le tre implementazioni. Chiama al suo interno il metodo `inorder_tree_walk()`.
- `select(i)`: esplora l'albero e restituisce il nodo in posizione *i*-esima rispetto ad un `inorder_tree_walk()`.
- `rank(key)`: esplora l'albero e restituisce la posizione rispetto ad un `inorder_tree_walk()` del nodo di chiave `key`.
- `min(node)`: restituisce il nodo con la chiave più piccola. Per costruzione è il successore più a sinistra.
- `max(node)`: restituisce il nodo con la chiave più grande. Per costruzione è il successore più a destra.
- `find(node, key)`: cerca il nodo con la chiave `key`. Se non viene trovato restituisce *None*.
- `successor(node)`: restituisce il successore del nodo `node`. Fa uso del metodo `min()`.
- `inorder_tree_walk(node)`: effettua un attraversamento ordinato dell'albero. Stampando gli elementi dal più piccolo al più grande.
- `size(node)`: Conta quanti nodi si trovano nei due sottoalberi destro e sinistro di `node`, compreso `node` stesso.

Albero AVL

Il singolo nodo dell'Albero AVL, `AVLNode` è stato implementato con 5 attributi: la chiave `key`, il puntatore al figlio destro `right`, il puntatore al figlio sinistro `left`, l'attributo `height` che tiene traccia del numero di archi nel percorso più lungo dal nodo alla foglia e, infine, l'attributo `size` che tiene traccia del numero totale di nodi nei sottoalberi destro e sinistro compreso il nodo radice. Quest'ultimo attributo è l'analogo del metodo `size(node)` dell'ABR che viene aggiornato solo quando modifico la struttura dell'albero, risparmiando così risorse computazionali.

Di sotto in dettaglio i metodi della classe:

- `insert(key)`: inserisce l'elemento di chiave `key` all'interno dell'albero nella posizione corretta. Effettua una rotazione appropriata per mantenere il `balance_factor` $\in [-1, 0, 1]$, utilizzando i metodi `right_rotate()` e `left_rotate()`.

- `delete(key)`: rimuove l'elemento di chiave *key* dall'albero effettuando in seguito una ristrutturazione tramite rotazioni per bilanciare i nodi in maniera corretta. Fa uso del metodo `successor()`.
- `print(key)`: metodo implementato per avere la stessa interfaccia fra le tre implementazioni. L'unica cosa che fa è chiamare il metodo `inorder_tree_walk()`.
- `select(i)`: esplora l'albero e restituisce il nodo in posizione *i*-esima rispetto ad un `inorder_tree_walk()`.
- `rank(key)`: esplora l'albero e restituisce la posizione rispetto ad un `inorder_tree_walk()` del nodo di chiave *key*.
- `get_height(node)`: restituisce l'altezza del nodo *node*.
- `get_size(node)`: restituisce la dimensione del nodo *node*.
- `get_balance(node)`: restituisce il fattore di bilanciamento del nodo *node*.
- `update(node)`: metodo di supporto. Ricalcola e aggiorna gli attributi `height` e `size` del nodo *node*.
- `right_rotate(node)`: effettua una rotazione a destra dell'albero.
- `left_rotate(node)`: effettua una rotazione a sinistra dell'albero.
- `min(node)`: restituisce il nodo con la chiave più piccola. Per costruzione è il successore più a sinistra.
- `max(node)`: restituisce il nodo con la chiave più grande. Per costruzione è il successore più a destra.
- `successor(node)`: restituisce il successore del nodo *node*. Fa uso del metodo `min()`.
- `inorder_tree_walk(node)`: effettua un attraversamento ordinato dell'albero stampando gli elementi dal più piccolo al più grande.

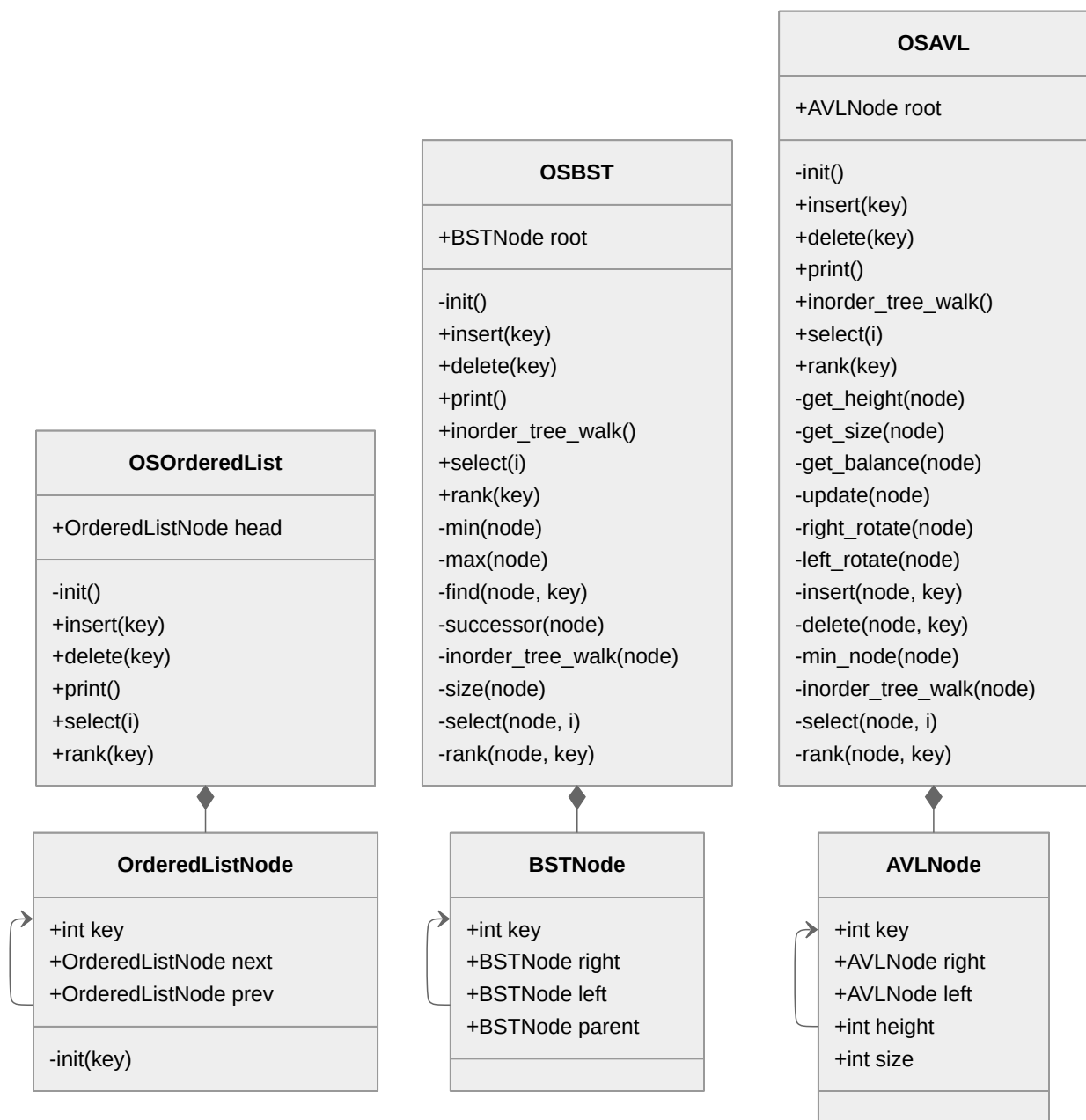


Figura 1: Diagramma UML delle tre implementazioni

3.3 Implementazione degli esperimenti

Gli esperimenti sono stati condotti confrontando le tre strutture dati implementate, `OSOrderedList`, `OSBST` e `OSAVL`, su 5 condizioni distinte, al variare della dimensione dell'input. In tutti i test il numero massimo di elementi inseriti è fissato a `size = 5000`, ad eccezione del test di *inserimento ordinato* il cui limite è fissato ad un terzo di quello degli altri ovvero `size =  $\lfloor \frac{5000}{3} \rfloor = 1666$` elementi. Ogni test procede eseguendo il controllo su insiemi di elementi che crescono di un passo `step = 50`, fino a che non si raggiunge la quantità `size`. Per ognuno di questi passi il test viene eseguito `repetitions = 10` volte. Il risultato di ogni ripetizione viene salvato in una lista ed in fine viene preso il valore mediano al fine di limitare le fluttuazioni date dagli inserimenti casuali che potrebbero

generare delle sequenze favorevoli o sfavorevoli. Il valore sarà il dato salvato nel file `.csv`.

- `test_casual_input`: misura il tempo di inserimento di sequenze di valori casuali distinti, estratti senza rimpiazzo dall'intervallo $[1, 10^7]$.
- `test_ordered_input`: misura il tempo di inserimento di sequenze ordinate in senso crescente $0, 1, 2, \dots$, caso particolarmente sfavorevole per il BST non bilanciato poiché degenera in una lista.
- `test_casual_deletion`: le strutture vengono prima popolate con una sequenza casuale di x elementi, dopodiché si elimina un sottoinsieme casuale di $\lfloor x/2 \rfloor$ elementi presenti nella struttura.
- `test_casual_ranking`: analogamente al test precedente, le strutture vengono pre-popolate con x elementi casuali; si misura poi il tempo necessario a eseguire l'operazione `rank` su $\lfloor x/2 \rfloor$ chiavi scelte casualmente tra quelle inserite.
- `test_casual_selection`: le strutture vengono pre-popolate con x elementi casuali; si misura il tempo per eseguire l'operazione `select` su $\lfloor x/2 \rfloor$ indici interi scelti uniformemente a caso nell'intervallo $[1, x]$.

Creazione dei grafici

Nel file `plot_results.py` sono state implementate alcune funzioni il cui scopo è scrivere, leggere e rappresentare i risultati sperimentali.

- `conv_pascal_case(file_name)`: si occupa di modificare il nome del file cambiandolo da *snake_case* a *Pascal Case* in modo che possa essere stampato assieme al grafico come didascalia.
- `save_csv(file_name, rows)`: si occupa di salvare i dati in un file `.csv` il cui nome corrisponde al tipo di test effettuato.
- `read_csv(file_name)`: si occupa di prendere il file `.csv` corretto e restituire una lista coi dati d'interesse che devono essere messi in grafico dalla funzione `plot()`.
- `plot(file_name)`: si occupa di creare i grafici dei dati e salvare le figure generate in formato `.pdf`.

4 Esperimenti e risultati

4.1 Scelta delle quantità

4.2 Presentazione dei risultati

In questa sezione si riportano tutti quanti i grafici dei risultati sperimentali. Si cerca di analizzarli per determinarne l'andamento. I grafici *a*, *b* e *c* di ogni sezione riportano le misurazioni effettuate per i singoli test, mentre il grafico *d* riporta tutte le misurazioni assieme al fine di facilitare il confronto.

Ingresso Lineare

Il risultato più significativo di questa batteria di test è rappresentato dalla figura 2d, nella quale si osserva chiaramente come, nel caso di inserimento di valori strettamente crescenti, la struttura basata su *Albero Binario di Ricerca* presenti un comportamento analogo a quello della lista ordinata. Nel caso della lista ordinata, ogni nuovo elemento deve essere inserito in coda e ciò richiede l'attraversamento di tutti gli elementi già presenti; di conseguenza, il costo di una singola operazione di inserimento cresce linearmente rispetto al numero di elementi memorizzati.

In modo analogo, un *Albero Binario di Ricerca* alimentato con valori strettamente crescenti degenera progressivamente in una struttura lineare, in cui ogni nodo possiede solamente il figlio destro. L'altezza dell'albero diventa quindi pari a n e ogni operazione di inserimento richiede l'attraversamento di tutti i nodi precedentemente inseriti, producendo anch'essa un andamento lineare rispetto alla dimensione della struttura.

Diverso è invece il comportamento dell'*AVL*. Grazie alle operazioni di bilanciamento e rotazione, l'altezza dell'albero rimane sempre logaritmica rispetto al numero di elementi memorizzati. Di conseguenza, anche nel caso peggiore di input ordinato, il costo delle operazioni di inserimento rimane pari a $O(\log n)$. Sebbene dai grafici tale andamento possa apparire quasi lineare, ciò è dovuto principalmente alla lenta crescita della funzione logaritmica e alle dimensioni relativamente contenute dell'input utilizzato nei test.

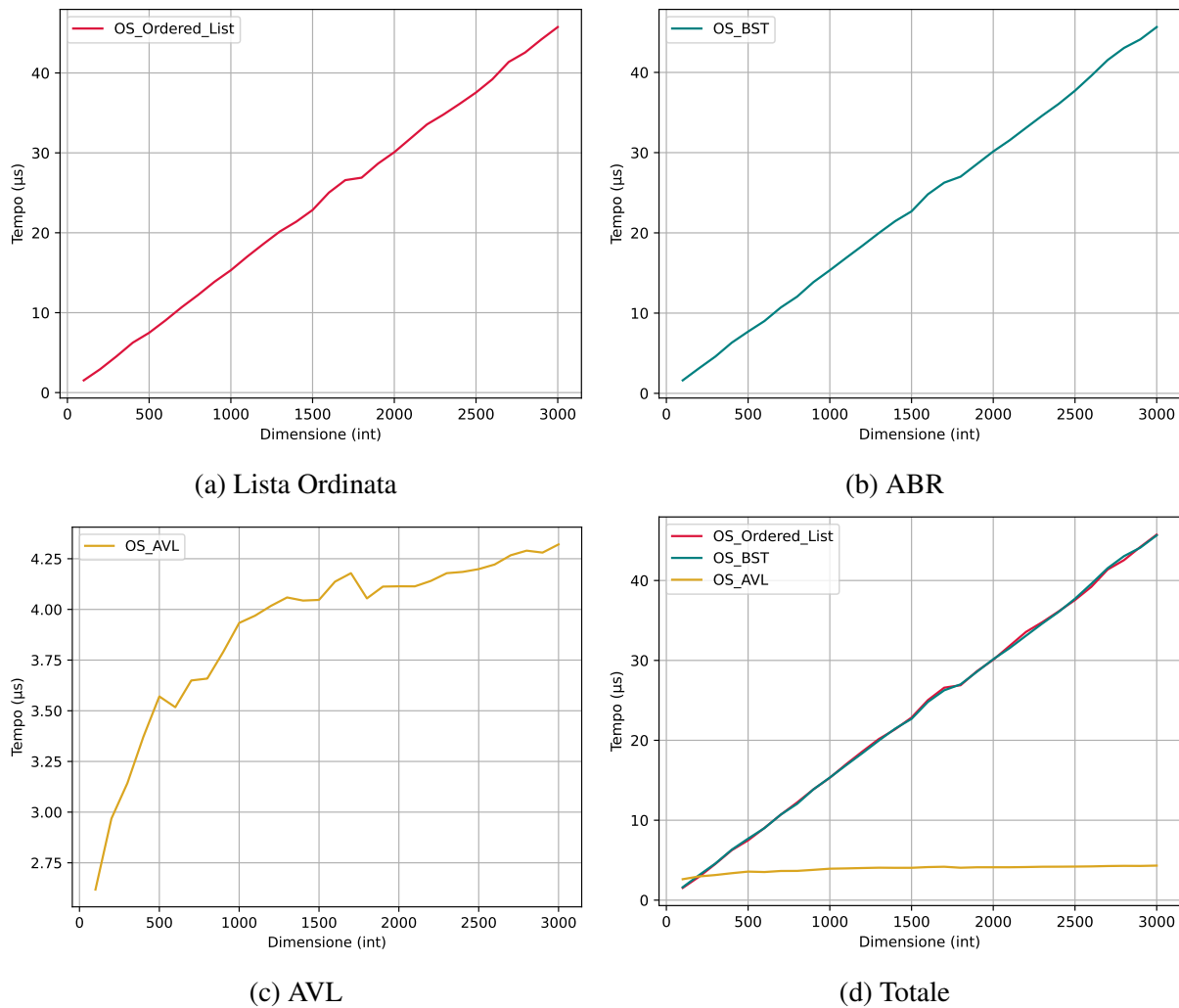
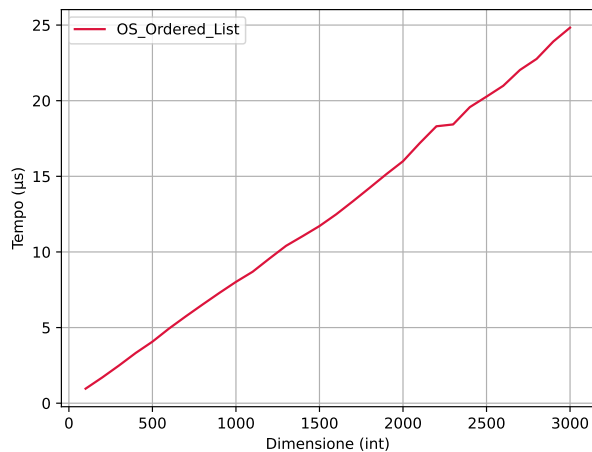


Figura 2: Andamento con ingresso lineare

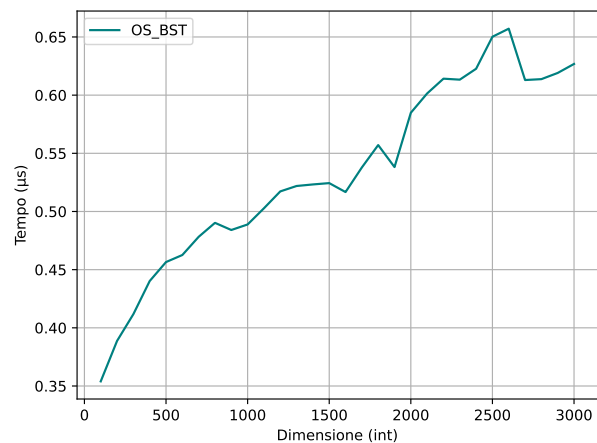
Ingresso Casuale

Nel test di inserimento casuale, riportato in figura 3d, sia l'AVL sia l'ABR mostrano un andamento molto simile e nettamente migliore rispetto alla lista ordinata. Questo comportamento è giustificato dal fatto che, con input casuali, l'ABR tende mediamente a mantenere una struttura sufficientemente bilanciata, ottenendo quindi un'altezza prossima a $\log n$. Di conseguenza, il costo medio delle operazioni di inserimento risulta anch'esso prossimo a $O(\log n)$.

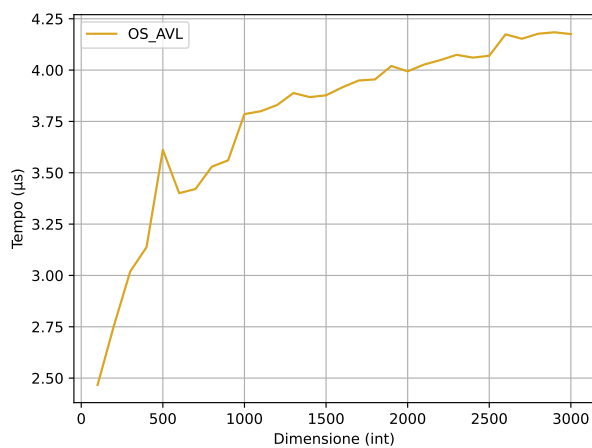
Si osserva inoltre come l'ABR risulti in alcuni intervalli leggermente più efficiente dell'AVL. Tale differenza è dovuta principalmente all'overhead introdotto dalle operazioni di bilanciamento dell'AVL, come l'aggiornamento delle altezze, delle dimensioni dei sottoalberi e le eventuali rotazioni. La lista ordinata, invece, continua a presentare un andamento lineare crescente, poiché ogni inserimento richiede mediamente l'attraversamento di una parte significativa della struttura.



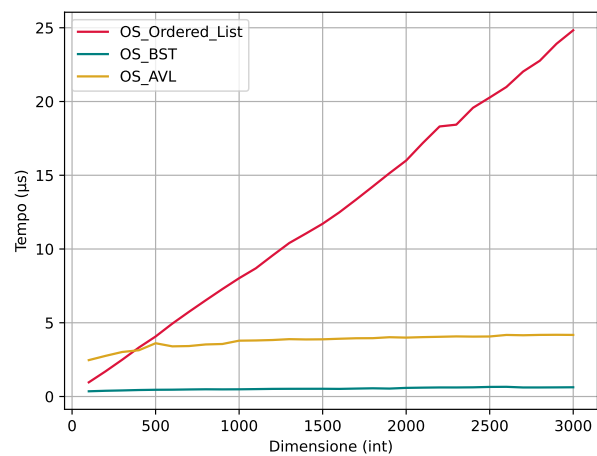
(a) Lista Ordinata



(b) ABR



(c) AVL



(d) Totale

Figura 3: Andamento con ingresso casuale

Cancellazione Casuale

Nel test di cancellazione casuale, riportato in figura 4d, sia l'*AVL* sia l'*ABR* mostrano un andamento simile e nettamente migliore rispetto alla lista ordinata. Questo comportamento è dovuto al fatto che le strutture vengono inizialmente popolate con valori casuali e risultano quindi mediamente bilanciate. In tali condizioni, le operazioni di ricerca del nodo da eliminare richiedono un tempo medio prossimo a $O(\log n)$.

La lista ordinata, invece, necessita di una scansione lineare per individuare ciascun elemento da rimuovere. Di conseguenza, anche il costo medio delle operazioni di eliminazione cresce linearmente rispetto alla dimensione della struttura, producendo una curva significativamente più accentuata rispetto a quelle ottenute con le strutture ad albero.

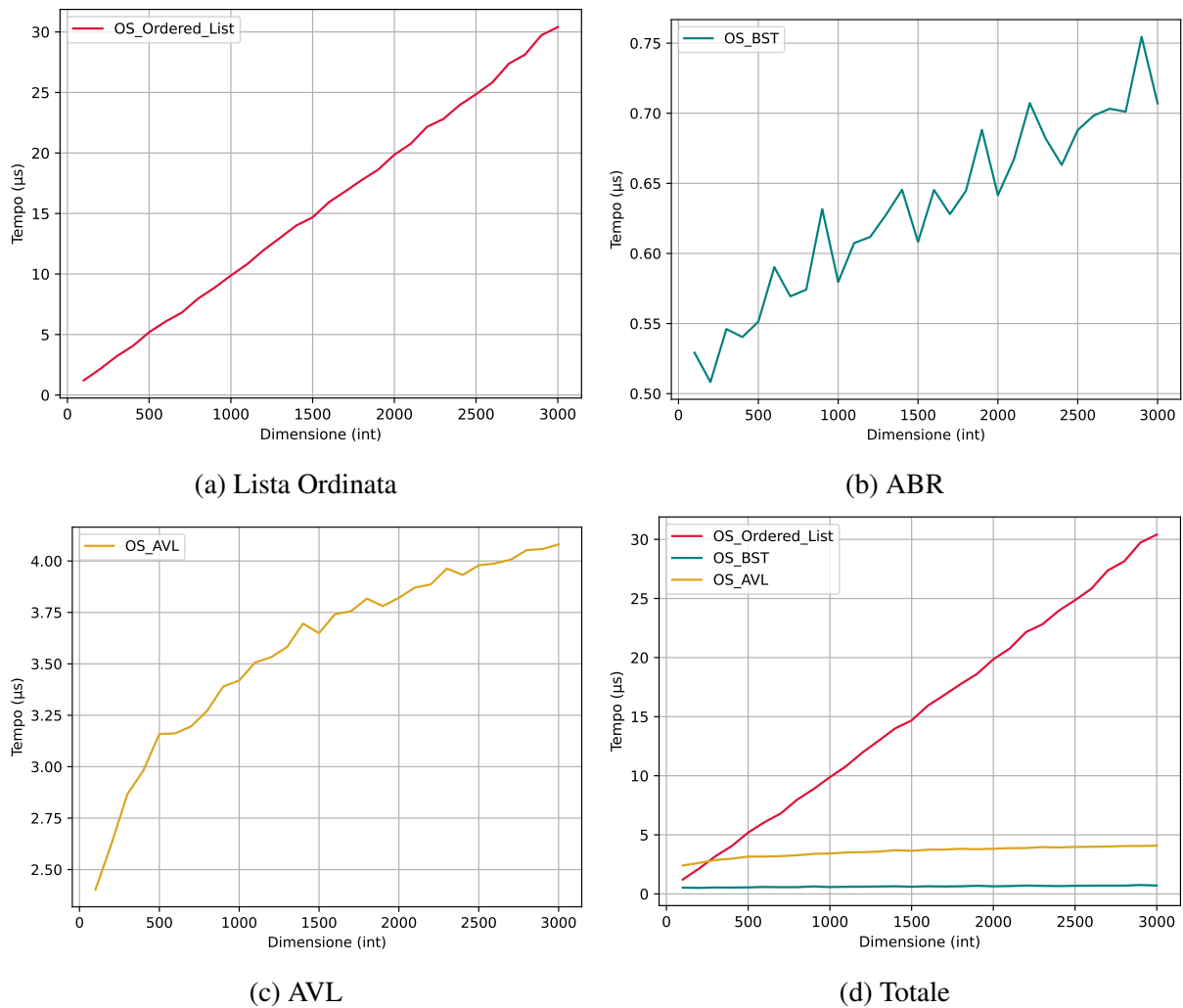
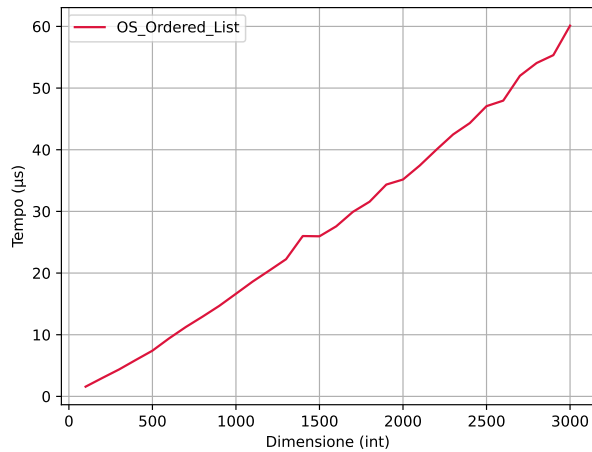


Figura 4: Andamento eliminazione casuale

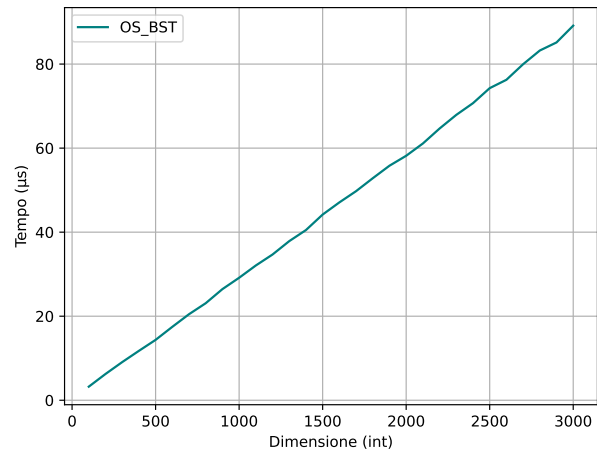
Ranking e Selezione Casuali

I test relativi alle operazioni di *rank* e *select*, illustrati nelle figure 5d e 6d, evidenziano in maniera particolarmente marcata la superiorità dell'AVL. In questa implementazione, infatti, ogni nodo mantiene l'informazione relativa alla dimensione del proprio sottoalbero; ciò consente di calcolare sia il rango di un elemento sia l' i -esimo elemento ordinato in tempo $O(\log n)$.

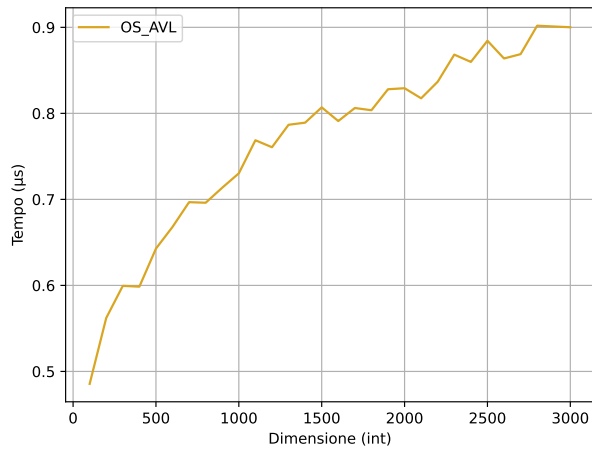
Nel caso dell'ABR, invece, le operazioni *rank* e *select* richiedono il ricalcolo ricorsivo della dimensione dei sottoalberi tramite la funzione `_size()`, aumentando significativamente il costo delle operazioni. Sebbene l'albero risulti mediamente bilanciato grazie all'inserimento casuale dei valori, il continuo ricalcolo delle dimensioni introduce un overhead aggiuntivo che rende le prestazioni sensibilmente peggiori rispetto a quelle ottenute con l'AVL.



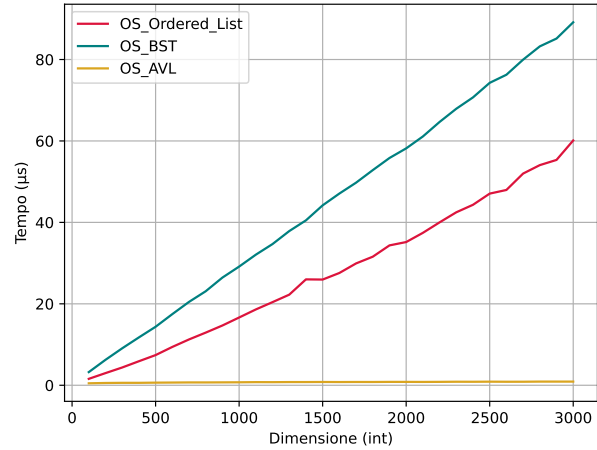
(a) Lista Ordinata



(b) ABR



(c) AVL



(d) Totale

Figura 5: Andamento ranking casuale

In particolare, nel test di *select* si osservano forti oscillazioni nel grafico relativo all'*ABR*. Questo fenomeno è dovuto principalmente alla natura casuale delle query effettuate e alla diversa struttura dei sottoalberi attraversati durante l'esecuzione dell'algoritmo. Infatti, la funzione `select()` può richiedere il calcolo della dimensione di sottoalberi molto differenti tra loro a seconda del valore richiesto; di conseguenza, il costo delle singole operazioni può variare sensibilmente tra un'esecuzione e l'altra. Tali oscillazioni risultano quindi compatibili sia con l'assenza di un meccanismo di bilanciamento rigoroso sia con il ricalcolo ricorsivo delle dimensioni dei sottoalberi.

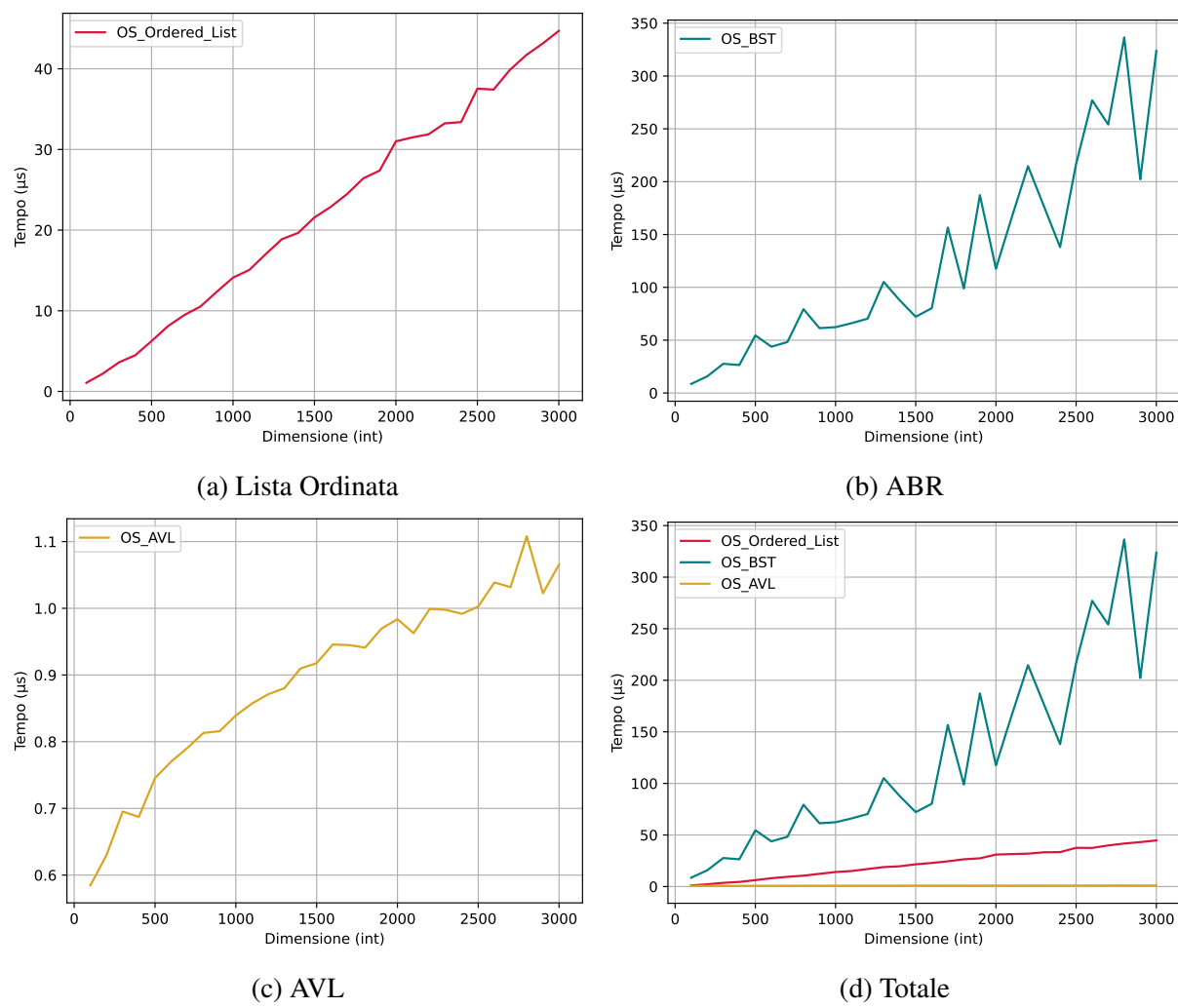


Figura 6: Andamento selezione casuale

5 Conclusioni